

Lists, strings, mutable and immutable variables

CMSC 104 Section 2
April 1, 2026

Administrative notes

Summary of the last two lectures:

Strings

Last
Wednesday

- Treating characters individually
- Manipulating strings
- Built-in methods
 - len, upper, lower, ...
- Indexing
 - Emphasize that you always start counting at 0
 - You can use negative index values to start counting from the end instead of the beginning
- Slicing
- Concatenating
- Repeating via “multiplication”

Lists

Monday

- Multi-valued data type
 - Mutable; can change individual list values
- Adding values (append, insert)
- Removing values (remove, del)
- len
- Indexing
 - You always start counting at 0
 - You can use negative index values to start counting from the end instead of the beginning
- Often used with a different type of “for” loop that processes each list element exactly once
- List elements don’t have to be the same type
- You can have lists of lists

A new kind of “for” loop

So what's all this nonsense about “mutable” and “immutable” variables?

“Mutable” means “can be changed”

“Immutable” means “cannot be changed”

In Python, the following variable types are immutable - they cannot be changed:

- Strings
- Ints
- Floats
- Booleans

The following variable types are mutable - they can be changed:

- Lists
- Dictionaries (we'll get to these later)

Strings as “unchangeable”

Here’s a short code sample:

```
s = "UMBC"
print(id(s))
s = "UMB"
print(id(s))
```

Let’s run that and see what happens

- Remember that “id(variable)” prints the location in memory where that variable’s value can be found
 - If you run this, you’ll get similar results but the actual locations in memory will be different

So: when we assigned the value “UMBC” to s, that was stored in one location in memory

But when we reassigned s the value “UMB” that was stored in another location in memory

We did not change the value of the original variable; we reassigned it and got a whole new location in memory. The value stored in the original memory location CANNOT be changed in place.

Why?

We're not going into details - you're not responsible for knowing this - but the answer is that research has shown your code actually runs a lot faster this way

- Making a memory location “unchangeable” means a lot of low-level functions don't have to be supported
- And lots of programmers write lots of code where a variable gets a value that never changes during program execution

Okay, but what about lists?

Here's another short code sample:

```
l = [1,2,3,4,5]
print(id(l))
for i in range(0,len(l)):
    l[i] += 1
print(l)
print(id(l))
```

See? The location in memory of the list, `l`, DID NOT CHANGE. The value of `l` was changed in place

Thus `l` is “mutable” - it can be changed in place.

Big whoop. What does that really mean?

More code:

```
def add_one(parameter):  
    for i in range(len(parameter)):  
        parameter[i] += 1  
    print(parameter)  
    # note that there is no return statement  
  
if __name__ == '__main__':  
    argument = [2, 4, 6, 8, 10] #this is where the magic happens  
    add_one(argument)  
    print(argument)
```

How was the value of “argument” changed without a return statement?

More Python “uniqueness”

When you call a function with an immutable argument - string, int, float, boolean - Python uses “call by value.”

- Python goes to the location in memory pointed to in the main program's symbol table
- Python makes a copy of that value, without changing the original
- Python places that copy in the function's symbol table.
- So nothing the function does can impact the original argument's value

When the argument is a mutable variable (e.g., a list)

Python works differently when the argument/parameter are lists.

- Python does not make a copy of the argument's value and place it in the function's symbol table.
- Instead, Python gets the address of the parameter from the main program's symbol table, and passes that address to the function, where it's placed in the function's symbol table
- The main program and the function are pointing to the exact same locations in memory
- Anything done to this variable by the function shows up the same way in the main program.

So now let's look at this code in detail:

```
def add_one(parameter):  
    for i in range(len(parameter)):  
        parameter[i] += 1  
    print(parameter)  
    # note that there is no return statement  
  
if __name__ == '__main__':  
    argument = [2, 4, 6, 8, 10] #this is where the magic happens  
    add_one(argument)  
    print(argument)
```

How was the value of “argument” changed without a return statement?

Because “parameter” and “argument” point to the same location in memory!!

Lists with elements of different types

There's nothing that requires all the elements of a list to be the same type

You can have a list that stores a student name as the first element, the student id as the second element, and then the student's grades as the rest of the elements:

```
l = ["Arsenault, Alfred", "ABC123", 89, 54, 33, 16, 22]
```

This is allowed, but it's not really common

- If you're going to process data like this, you're more likely to use (spoiler alert) a ***dictionary***

Multi-dimensional lists

You can have a list, whose elements are each lists

Let's start with a table, like the one here:

<https://worldathletics.org/competitions/world-athletics-championships/world-athletics-championships-tokyo-2025-7190593/medaltable>

You can encode that in Python as a two-dimensional list

```
medal table = [  
    ["United States", 10, 5, 5],  
    ["Kenya", 7, 2, 2],  
    ["Canada", 3, 1, 1],  
    ["Netherlands", 2, 2, 2],  
    ["Botswana", 2, 2, 2]  
]
```

Two-dimensional lists

The lists we've dealt with up to now are called "one-dimensional lists" - they are lists whose elements are individual variables

"Medal_table" is a two-dimensional list - a list, whose elements are one-dimensional lists

Terminology: two dimensional lists consist of "rows" and "columns"

The "row" index always comes first, then the "column" index

```
medal_table[0][1]
```

Refers to the number of gold medals won by the United States - it's the second element (column) in the first row

Handling two-dimensional lists

Python makes dealing with a row very easy,

```
medal_table[0]
```

Is the entire first row in the 2-d list.

Adding another row is trivial:

```
medal_table.append([ "Spain", 2, 0, 1])
```

But columns are difficult

You can't directly add a column to each row in a 2-d list. You have to add an item to each individual row and then write a loop to do this to all rows

- Each row in medal_table shows the number of gold, silver and bronze medals won by that country, but it doesn't show the total number of medals. So let's add that

- To do this in one row:

```
total = medal_table[0][1]+medal_table[0][2] + medal_table[0][3]
medal_table[0].append(total)
```

- To do it in all rows:

```
for i in range(1,len(medal_table)):
    total = medal_table[i][1] + medal_table[i][2] + medal_table[i][3]
    medal_table[i].append(total)
```


You can have three-d, four-d, etc. lists in Python

Those are fairly rare, because they get hard to work with

You'd usually start using other data structures, like lists of dictionaries, instead

Dictionaries

You weren't confused enough. Now we're going to introduce another type

Lists are fine. Useful, powerful. But we need more in order to do the work we need to do.

Sometimes we have data of different types that all relate to the same record. And we want to keep that together, rather than in different variables.

E.g, here's a fake "personnel record"

A	B	C	D	E	F	G
LastName	FirstName	Employee_ID	Department	College	Rank	Salary
Arsenault	Alfred	ABC123	EECS	COEIT	Adjunct	\$1/hr

How do we process the data in this record?

Option 1: lots o' variables:

```
LastName = "Arsenault"  
FirstName = "Alfred"  
EmpID = "ABC123"...
```

Clearly a suboptimal way of doing things. Managing all those variables is going to drive the programmer up a wall. Mistakes are highly likely to occur.

Option 2: lists

```
emp = ["Arsenault", "Alfred", "ABC123", "EECS", "COEIT", "Adjunct", 1]
```

Better, but still less than optimal. The programmer has to remember that `list[0]` is for the last name, `list[6]` is for the hourly salary, etc. Again, mistakes are quite possible

The answer: dictionaries

So, Python provides a built-in data type/data structure called a ***dictionary*** that is made to handle situations just such as this one.

The basic syntax:

```
Dictionary_name = {  
    Key_1:value_1,  
    Key_2:value_2,  
    ...    Key_n:value_n  
}
```

In this particular case:

illustration of how dictionaries work

```
employee_record = {  
    "last_name": "Arsenault",  
    "first_name": "Alfred",  
    "emp_id": "ABC123",  
    "dept": "EECS",  
    "college": "COEIT",  
    "rank": "Adjunct",  
    "hourly_pay": 1  
}
```

Keys vs values:

- The things to the left of the colons are called “**keys**” Those describe what it is we’re storing here
- The things to the right of the colons are the “**values**” associated with each key.

When to use a dictionary

Dictionaries are used when you want to keep together data of different types that all relate to the same object, item or record

- They don't have to be of different types; they could all be the same, but that's not common

The keys in a dictionary variable explicitly tell the programmer what the heck this particular thing is - "last_name", "emp_ID", etc.

- You the programmer don't have to remember, like you would with a list

Keys and Values

Keys

- Must be unique. You can't have a key used twice
- Must have a unique value
- Must be of an immutable type
 - String, float, int, boolean
 - You CANNOT have keys that are lists or dictionaries

Values

- Need not be unique. Different keys can have the same values
- Can be of any type
 - Values can be lists
 - Values can be dictionaries - you can put a dictionary inside another dictionary